

Mira — Component Level Design Document

1. Purpose & Scope

This CLDD documents **how** Mira is architected and **why** each major technical path was chosen over the alternatives considered — complementing, not duplicating, the two companion documents:

- **Mira — Problem Statement & PRD:** the *what* and *why it matters* — problem, users, scope, user stories, success metrics.
- **Mira — Architecture & Node Reference:** the *implementation record* — every n8n node, parameter, and full prompt text, in execution order.

This HLD provides: system context, component boundaries, data design, alternative approaches considered covering the major product/architecture decisions made during development, with the rejected option and the evidence that drove the choice. Where that evidence comes from the build's own test artifacts (test cases, the fine-tuning scripts, the DeepEval suite, the workflow JSON).

2. System Context

Actor / System	Relationship to Mira
PM/TPM (primary user)	• Sends requests via the web interface; receives every response in the web interface “ask-mira.html”. • approves/denies new-risk register writes through shared Slack channel <code>mira-updates</code>.
External caller (webhook)	POSTs {input, sessionId} to the workflow's webhook path; receives a synchronous text response
Google Drive	Source of truth for project description, timeline, risk register, and task board documents (read-only from Mira's side)
Google Sheets	• Live system-of-record for the Risk Register, Timeline and Task Board — read by every risk/status request • Risk register updates - written only for approved new risks
Pinecone	Vector store (<code>mira-pineconeindex</code>) holding chunked, embedded copies of the Drive documents — the RAG layer, rebuilt on a schedule.
OpenAI	LLM (<code>gpt-4o-mini</code>) for all 5 agents; embeddings (768-dim) for ingestion and retrieval
Slack	Output channel for every response, the human-in-the-loop approval surface, and the error-alert channel (<code>#errors</code>)
LangSmith	Tracing and offline evaluation of every agent run

Trust boundary: Mira operates entirely within Nexora's own tooling (n8n instance, Google Workspace, Slack workspace) — there is no public-facing surface other than the web interface “ask-mira.html” (connected to webhook endpoint itself), which is unauthenticated in the current build (see Open Technical Risks).

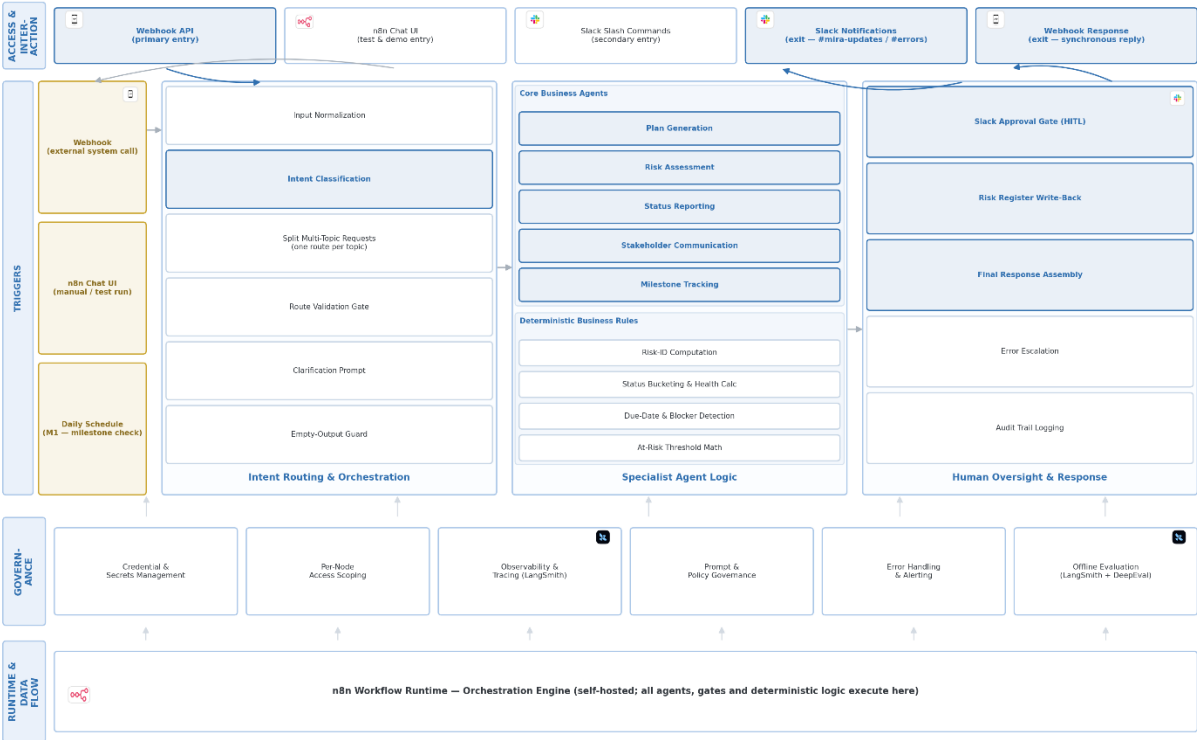
Mira — Component Level Design Document

3. System Architecture

Mira-System-Architecture depicts the following architectural elements: entry/exit points, orchestration, specialist-agent business logic, human oversight, governance.

Mira — System Architecture

Agentic workflow platform: entry points, orchestration, business-logic agents, human oversight & exit points



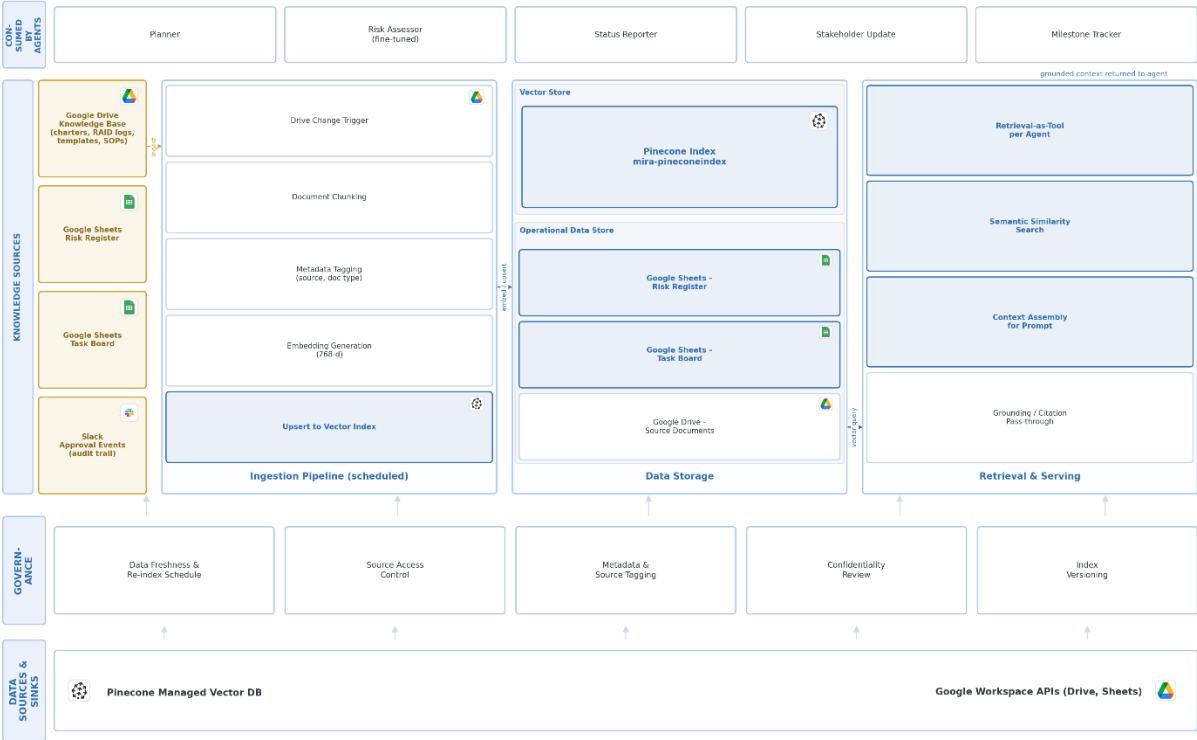
4. Data Architecture

Mira-Data-Architecture depicts the following architectural elements: Google Drive knowledge base, ingestion pipeline, Pinecone vector store, retrieval-as-tool serving.

Mira — Component Level Design Document

Mira — Data Architecture

Knowledge ingestion, vector storage & retrieval pipeline that grounds every agent's responses



5. System Flow – Component Layers

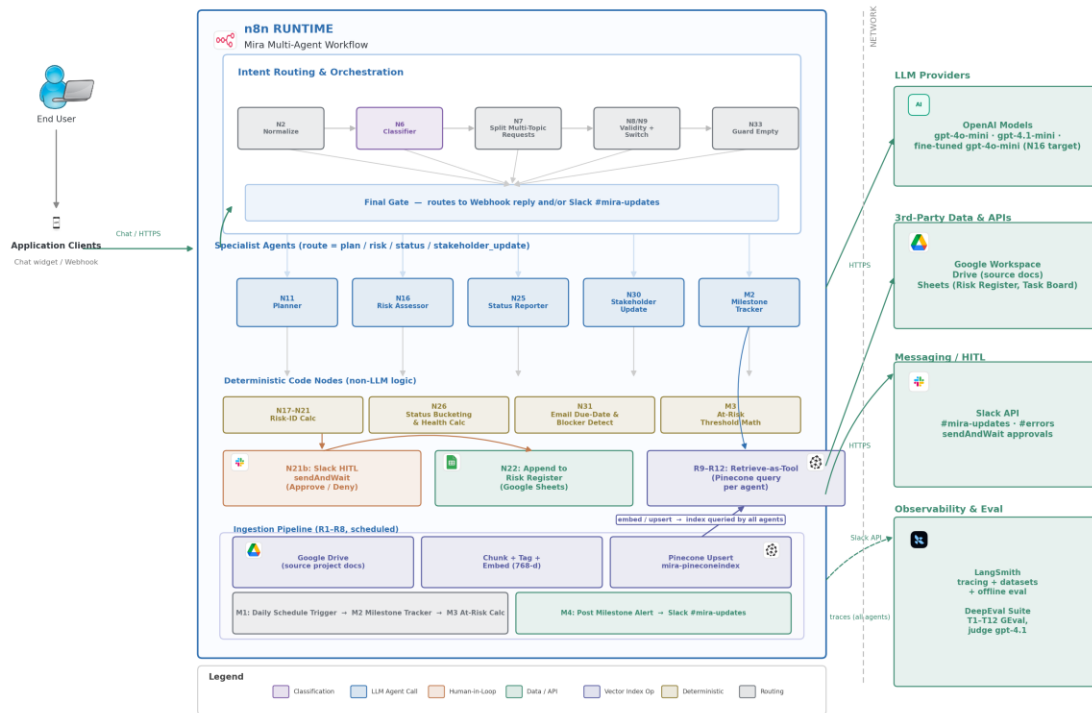
Workflow Design

The n8n Runtime container nests Intent Routing & Orchestration, the Specialist Agent Pool, the Deterministic Code layer, the HITL Slack approval gate, and Retrieve-as-Tool — with external LLM, Data/API, Messaging, and Observability systems drawn outside the network boundary.

Mira — Component Level Design Document

Mira — Agentic Workflow SystemFlow

n8n multi-agent runtime, drawn as a platform solution diagram



Two independently-schedulable n8n workflows implement Mira. These two n8n workflows are mapped to Orchestration Pattern and RAG/Knowledge Retrieval:

1. **Ingestion pipeline** — keeps the vector index synchronized with the Drive source of truth. Pulls project files (description, timeline, risk register, task board) from Google Drive on a schedule, chunks them (character splitter, 250-token overlap), tags each chunk's source type, embeds with OpenAI (768-dim), and upserts into a namespaced Pinecone index — idempotently (wipes and rebuilds the namespace rather than accumulating duplicates).
2. **Retrieval pipeline** — the live, request-serving multi-agent system: a Router/Dispatcher Classifier Agent dispatches to one or more of five specialist agents (Planner, Risk Assessor, Status Reporter, Stakeholder Update, and the scheduled Milestone Tracker), each grounded via its own Pinecone retrieve-as-tool call (the LLM decides when and what to query). Non-LLM Code nodes are deterministic nodes that re-derives counts, due-date math, and (for Risk) the next risk_id, rather than trusting the LLM's own arithmetic. Further to action, appending a new risk; is additionally gated behind a Slack human-approval step.

Every agent's execution is traced to LangSmith for observability and results can be utilized for offline evaluation.

Mira — Component Level Design Document

How Mira applies it

Concept	How Mira applies it
Multi-Agent System Design	5 specialist agents (Planner, Risk Assessor, Status Reporter, Stakeholder Update, Milestone Tracker) + 1 Orchestrator, each a distinct n8n-agent node with its own system prompt, tools, and output schema
Orchestration Pattern	Router/Dispatcher (Classifier Agent + Switch node), extended for multi-intent routing
Prompt Engineering	Structured system prompts per agent enforcing strict JSON schemas, step-by-step reasoning pipelines, and explicit "gatekeeper" sufficiency rules.
Data Grounding	Every agent's tool retrieves only from the live Pinecone index (metadata-filtered per agent), never from the model's own memory; further deterministic Code nodes like N17d, N26d additionally re-verify LLM output against the live Google Sheets before anything is returned
Evaluation & Observability	LangSmith tracing (enable via the environment variables set in local windows) + 4 offline evaluators (Hallucination, Assertions, Answer Relevance, Bias & Fairness) against a purpose-built dataset; a separate DeepEval suite for the fine-tuned agent Document available at \resourcefiles\mira_Ds_v5.csv
Fine-Tuning (extended)	Risk Assessor Agent (N16) fine-tuned on 9 examples. Document available at \docs\Mira-PipelineOutputsforFineTuning.xlsx
RAG / Knowledge Retrieval (extended)	The entire grounding mechanism for all five agents (see above) Documents available (uploaded to Google Drive for Mira) at \resourcefiles
Human-in-the-Loop (extended)	Slack sendAndWait double-approval gate before any new risk is written to the register

Model tiering (as-built): Classifier, Planner, Risk Assessor, and Milestone Tracker run on gpt-4o-mini (Risk Assessor also has a fine-tuned gpt-4o-mini variant, see PRD Section: Evaluation Technical Detail below). Status Reporter and Stakeholder Update run on gpt-4.1-mini (maxTokens 8192) — both return larger, more structurally demanding JSON payloads that benefited from the larger model.

6. Component Design Summary

Each row is a deployable unit of responsibility inside the two n8n workflows. Node IDs point to the document “**Workflow Architecture & Node Reference** document” for full parameters and prompt text.

Component	Responsibility	Interface (in → out)	Node reference
Ingestion Pipeline	Keeps Pinecone in sync with the Drive source-of-truth documents: wipes the namespace, re-downloads, chunks, tags each chunk with a source metadata field by filename pattern, embeds, and re-inserts	Schedule trigger (1 min) → Google Drive files → Pinecone upserts	R1–R8
Orchestrator / Classifier Agent	Single entry point for every request; classifies intent(s) and supports multi-intent fan-out ({routes:[], reason})	Normalized user/webhook input → one or more route tags	N6, N6 Simple Memory
Planner Agent	Produces/updates the project plan from Drive context and conversation memory	Route = plan → retrieve-as-tool query to Pinecone → plan text/table	N11–N15
Risk Assessor Agent	Classifies a request against the risk register: return	Route = risk → retrieve-	N16, gpt-4o-mini (fine-

Mira — Component Level Design Document

Component	Responsibility	Interface (in → out)	Node reference
	existing risks (filtered/counted) or draft a new risk entry	as-tool query to Pinecone + risk-register-search tool → structured risk JSON	tuning target)
Deterministic Guardrail — Risk	Non-LLM re-check of N16's output against the live Risk Register sheet before anything is shown or written; the fine-tuned model raised the pass rate here, it didn't replace the check	Risk Assessor JSON + live Sheets read → verified/corrected JSON	N17, N18, N20, N21 (real node IDs; parse → sufficient-detail gate → new-risk check → deterministic risk_id build)
Status Reporter Agent	Summarizes sprint/task status by category	Route = status → retrieve-as-tool query to Pinecone → status draft	N25, gpt-4.1-mini
Stakeholder Update Agent	Drafts a sprint stakeholder-update email (subject, progress summary, next steps) from live task/timeline data	Route = stakeholder_update → shares N25's Pinecone retrieve-as-tool → prose JSON draft	N30 (gpt-4.1-mini)
Deterministic Guardrail — Stakeholder Update	Non-LLM check that every in-scope task's due_date is compared to today; assembles the final email so a genuinely overdue/blocked task can never be silently omitted from the prose — added after a real bug caught via LangSmith tracing where the agent's own prose missed an overdue task	Stakeholder Update draft → verified email text	N31
Deterministic Guardrail — Status	Non-LLM recompute of at-risk items and sprint totals against the live Task Board sheet	Status draft + live Sheets read → verified status matrix	N26 (Format Status Report — buckets tasks and computes overall_health in code, not the LLM)
Milestone Tracker Agent	Daily proactive check for upcoming/missed milestones, independent of user requests	Daily schedule trigger → retrieve-as-tool query to Pinecone → Slack digest	M1–M4S
HITL Approval Gate	The only mutating path in the system: a new risk-register row requires explicit Slack approval before it is written	New-risk JSON → Slack sendAndWait → Approve/Deny → conditional Sheets append	N21b, N21c, N22
Persistence Layer	System-of-record reads/writes: Risk Register and Task Board (Google Sheets, read-heavy + gated writes); Pinecone (derived index, rebuilt not authored)	Sheets read/append nodes; Pinecone query/upsert nodes	N22 (Sheets append), R8 (Pinecone upsert) — reads happen via each agent's own retrieve-as-tool call, not a separate persistence node
Observability	Every agent invocation traced to LangSmith for tracing and offline evaluation (Hallucination, Assertions, Answer Relevance, Bias & Fairness, Conciseness evaluators)	Agent LLM calls → LangSmith project	LangSmith config
Fine-Tuning Pipeline	Offline: fine-tunes the Risk Assessor's base model on corrected bug-fix examples, independent of the live workflow	training_data_risk_assessor.jsonl → OpenAI fine-tune job → model ID pasted into N16	fine_tune_risk_assessor.py, test_risk_assessor.py with nine Eval cases T1–T9

Mira — Component Level Design Document

7. Data Design

Pinecone index

- Index `mira-pineconeindex`, 768-dim embeddings (OpenAI embeddings model), single namespace, fully rebuilt (not incrementally patched in this MIRA iteration) on every ingestion run — the pipeline deletes the namespace's vectors before re-inserting, so the index is always a clean mirror of Drive, never a place old/stale chunks can accumulate.
- Each chunk carries a source metadata field, set by filename pattern at ingestion time (e.g. project description, risk register export, task board export). This is what lets each agent's retrieve-as-tool call filter to only the document types relevant to its job — the Risk Assessor doesn't retrieve status-report chunks, and vice versa.
- Retrieval is agent-initiated ("retrieve-as-tool"), not a static pre-fetch: each specialist agent decides at runtime whether and what to query, with its own `source` filter and top-K.

Google Sheets — Risk Register

Field	Notes
Risk ID	stable identifier, referenced by the deterministic guardrail
Description	free text
Category	used for filtered/counted queries ("how many schedule risks")
Severity / Likelihood	structured, drives risk scoring
Status	open/mitigated/closed
Owner	
Date added	written only on approved HITL writes

Read by N17b on every risk-related request (the ground truth the deterministic guardrail checks the LLM's output against); appended to by N22, and only after Slack approval.

Google Sheets — Task Board

Field	Notes
Task ID	
Title / Description	
Category	filter dimension for Status Reporter queries
Sprint	used by the deterministic guardrail to recompute sprint totals
Due date	used to recompute at-risk status live, rather than trusting the LLM's claim
Status	not started/in progress/done/at risk

Read by N26b on every status request; never written to by Mira — status reporting is read-only.

Mira — Component Level Design Document

Google Sheets — Timeline

Field	Notes
Milestone ID	
Phase Number / Phase Name	groups milestones into project phases; read by the Planner Agent to structure the roadmap output
Milestone Name	
Planned Start Date	
Planned End Date	used by the deterministic guardrail to recompute overdue / impending-risk status live, rather than trusting the LLM's own date judgment
Status	not started / in progress / done — a Done milestone is always excluded from the at-risk check, regardless of its dates
Owner	
Dependency Task IDs	semicolon-separated Task Board IDs; each is cross-checked against the Task Board's own Due date/Status at evaluation time to catch a blocking task that's still open or already overdue

Read by R9 (Planner Context), R11 (Status Context — shared with the Stakeholder Update Agent), and R12 (Milestones Context) on every relevant request; never written to by Mira — timeline tracking is read-only, same as the task board.

Data routing against components

- **Classifier → downstream agents:** {routes: string[], reason: string} — supports one request fanning out to multiple agents.
- **Risk Assessor → guardrail (N17d):** structured risk JSON (existing-risk matches or a draft new-risk object) that the guardrail re-derives independently from the live sheet and reconciles.
- **Status Reporter → guardrail (N26d):** structured status JSON (per-category summary, at-risk flags, sprint totals) that the guardrail recomputes from the live Task Board rather than trusting verbatim.
- **New-risk → HITL:** the exact row to be appended, rendered into the Slack `sendAndWait` approval message so the approver sees precisely what will be written.

8. Alternative Approaches Considered

The major technical/product decisions made during development, the alternative that was set aside, why, and the evidence that drove the call — drawn from the build's own artifacts (workflow JSON testing with respect to planned use cases, evaluation suite, fine-tuning scripts).

1. Retrieval: agent-initiated "retrieve-as-tool" vs. static pre-fetch RAG

- **Chosen:** each specialist agent calls Pinecone as a tool at its own discretion, with its own `source` metadata filter and top-K.
- **Alternative considered:** pre-fetching a fixed set of chunks before the agent runs and stuffing them into the prompt context.
- **Why rejected:** a pre-fetch has to guess relevance before the agent has classified the actual sub-question; retrieve-as-tool lets the agent decide it needs (or doesn't need) grounding, and query differently for "how many schedule risks" vs. "show me risk R-014".

Mira — Component Level Design Document

- **Evidence:** the differing source filters and topK per agent (R9–R12) only make sense because retrieval is scoped per call, not shared across a single pre-fetch step.

2. Deterministic guardrail layer vs. prompt-only grounding

- **Chosen:** non-LLM Code nodes (N17d, N26d) that independently re-derive the answer from the live Google Sheets and reconcile it against the LLM's output before anything reaches the user.
- **Alternative considered:** relying on prompt instructions alone ("always check the sheet, never hallucinate a risk") to keep the Risk Assessor and Status Reporter grounded.
- **Why rejected:** prompt-only grounding is a probabilistic control on a system explicitly built because manual risk assessment and status reporting were already inconsistent — replacing one inconsistent process with an LLM that occasionally still drifts doesn't solve the stated problem always. A deterministic re-check guarantees the register/board is the final word, regardless of what the LLM claims.
- **Evidence:** N16 needed tuning specifically to fix a routing bug the deterministic layer caught in testing (drafting a "new" risk that already existed in the register) — i.e., the guardrail surfaced the failure mode prompt tuning alone hadn't eliminated.

3. Human-in-the-loop scope: gate only new-risk writes, not every response

- **Chosen:** Slack `sendAndWait` approval only on the one mutating action — appending a new row to the Risk Register.
- **Alternative considered:** gating every agent response (plans, status reports, existing-risk lookups) behind human approval, matching the original manual process where a PM reviewed everything.
- **Why rejected:** approving every read-only response would reintroduce the same time cost the project set out to remove (the 1–2 hour status-compilation and 3–4 hour planning problem); gating is only needed where Mira can permanently change the system of record.
- **Evidence:** N21b's approval payload renders exactly the new-risk row that would be written, scoping the review to the one place a wrong answer has a lasting effect.

4. Dual entry point: chat widget + webhook, rather than one

- **Chosen:** N1 accepts both the n8n chat widget and an external webhook (`{input, sessionId}`), normalized to a common shape at N2.
- **Alternative considered:** chat-widget-only.
- **Why rejected:** a webhook lets other systems (or later, a proper front-end) call Mira programmatically without a person driving the n8n chat UI, at negligible extra cost since both paths converge immediately after normalization.
- **Evidence:** both entry points share every downstream node from N2 onward.

5. Local n8n instance vs. cloud-hosted n8n

- **Chosen:** locally run n8n for this build.
- **Alternative considered:** n8n Cloud.

Mira — Component Level Design Document

- **Why rejected:** local setup gave direct control over credentials, webhook exposure, and the LangSmith/Slack/Pinecone integration wiring during iterative development, without the account/billing overhead of a hosted plan for a capstone project.
- **Evidence:** the setup screenshots (see the Workflow Architecture & Node Reference doc) document the local install and CLI commands used to bring the instance up.

6. Bare vague-risk queries

- **Chosen:** Vague Prompts seeking for risks information, for example: “risks?”.
- **Alternative considered:** System Prompt – Prompt Re-engineering. Needs a prompt-engineering correction to N16's Step-0 intent classification.
- **Why rejected:** A one-word query like "risks?" scores 0.11 on the Vague Statement Handling metric — N16 dumps the full register instead of asking for scope.
- **Evidence:** Testing confirmed fine-tuning does not fix this failure class. The setup screenshots present the scenario. The change tightened the vague-vs-broad distinction, not another fine-tuning pass.

9. Technology Stack

Layer	Choice	Specifics
Orchestration platform	n8n	Local instance
LLM	OpenAI	gpt-4o-mini for Classifier, Planner, Risk Assessor and Milestone Tracker; gpt-4.1-mini (maxTokens 8192) for Status Reporter and Stakeholder Update, whose larger structured-JSON payloads needed the bigger model
Fine-tuned model	OpenAI fine-tuning API	gpt-4o-mini-2024-07-18, 3 epochs, suffix mira-risk-assessor, applied to the Risk Assessor's chat model node only
Embeddings	OpenAI embeddings	768-dim, used identically at ingestion and retrieval time
Vector store	Pinecone	Index mira-pineconeindex, single namespace, wipe-and-rebuild ingestion
Persistence (system of record)	Google Sheets	Risk Register, Task Board — read on every relevant request, written only via the HITL-gated path
Source documents	Google Drive	Project description, timeline, risk register export, task board export
HITL / notification channel	Slack	sendAndWait approval node; mira-updates output channel; #errors alert channel
Memory	n8n Simple Memory (buffer window)	Per-session conversation memory on the Classifier
Observability	LangSmith	Tracing on every agent call; offline evaluation datasets and evaluators (Hallucination, Assertions, Answer Relevance, Bias & Fairness, Conciseness)
Fine-tuning tooling	Python + OpenAI SDK	fine_tune_risk_assessor.py (train/upload/poll/test/rollout)
Evaluation tooling	DeepEval	test_risk_assessor.py (renamed from the -before-fine-tuning suffix), custom GEval metrics, judge model gpt-4.1
Deterministic verification	n8n Code nodes (JavaScript)	N17–N21 (risk: sufficient-detail gate + risk_id calc), N26 (status: bucketing + health), N31 (stakeholder update: due-date check) — no LLM involved in any of the three final checks

Mira — Component Level Design Document

10. Deployment & Environment

- **Runtime:** n8n installed and run locally (see the Workflow Architecture & Node Reference doc's Setup Screenshots section for the install/CLI commands actually used).
- **Credentials:** OpenAI, Pinecone, Google (Drive + Sheets, OAuth), Slack, and LangSmith credentials are all stored in n8n's own credential store, scoped per-node rather than hardcoded in prompts or Code nodes.
- **Two independently-triggered workflows:** the Ingestion pipeline runs on a 1-minute schedule trigger (R1); the Retrieval pipeline is request-triggered (chat widget or webhook) plus its own daily schedule branch for the Milestone Tracker (M1) — they do not share a trigger or a run context.
- **Environment variables / config surface:** Pinecone index name, Google Drive folder ID, Sheets document IDs, Slack channel IDs, and the LangSmith project name are the main environment-specific values.
- **Network/security posture:** everything runs inside Nexora's own n8n instance, Google Workspace, and Slack workspace — no data leaves those boundaries except to OpenAI (LLM/embeddings), Pinecone (vector storage), and LangSmith (traces). The one exception is the webhook entry point, which is currently unauthenticated (flagged under Open Technical Risks below).

11. Functional Requirements

Each specialist agent's output is validated by a downstream parser and, for Risk/Status, re-verified against the live Google Sheets data; adding a deterministic layer to solutioning.

Agent	Input	Output schema (key fields)
Planner (N11)	User's plan request + Pinecone-retrieved project description/timeline/task-board chunks	sufficient_detail (bool), answer (grounding summary), missing_info[], phases[] — each with phase_name, activities[], milestones[], deliverables[], timeline_estimate
Risk Assessor (N16)	User's risk request + Pinecone-retrieved register/task-board/timeline chunks	intent (report question both), answer, requested_count, risk_list_heading, sufficient_detail, missing_info[], risks[] (existing, with risk_id, category, risk, impact, mitigation, matched_entity), new_risks[] (unmatched, risk_id:"")
Status Reporter (N25)	User's status request + Pinecone-retrieved task-board/timeline chunks	sufficient_data (bool), no_actionable_request, answer, report_scope, requested_categories[], tasks[] (task_id, task, status, due_date, reason), upcoming_milestones[] (milestone_id, milestone) — N25 emits a flat tasks[] list and category scope only; a downstream Code node (N26) deterministically buckets into done/in_progress/to_do/blocked/at_risk (comparing due_date to today in real code, not LLM judgment) and computes overall_health (Green Amber Red)
Milestone Tracker (M2)	Today's date + Pinecone-retrieved timeline/task-board chunks (daily, no user input)	at_risk[] (milestone_id, milestone_name, planned_end_date, status, reason, suggested_action), on_track_count, summary
Stakeholder	User's stakeholder-update request + Pinecone-retrieved task-board/timeline	sufficient_data (bool), insufficient_data_message, subject, progress_summary, next_steps, tasks[] (task_id, task,

Mira — Component Level Design Document

Agent	Input	Output schema (key fields)
Update (N30)	chunks (shares N25's retrieval tool)	status, due_date, reason) — N30 writes prose fields only; a downstream Code node (N31) deterministically checks each task's due_date and assembles the final blocker/overdue sentence and email text, the same guardrail pattern used for Status Reporter

12. Non-Functional Requirement Implementation Detail

Non-Functional Requirements detailed below:

Requirement	Node-level implementation detail
Grounding / anti-hallucination	Every agent prompt carries a "NO HALLUCINATED FACTS" core principle; retrieval is metadata-filtered per agent's domain; a deterministic Code-node layer (N17d for risk, N26d for status) independently re-derives figures from the live Google Sheets rather than trusting the LLM's own arithmetic or recall
Auditability	Every final response — plan, risk matrix, status report, clarification, or error — is posted to one Slack channel (mira-updates), creating a single running record of every Mira interaction regardless of entry point (chat vs. webhook)
Governed writes	The one mutating action (appending a new risk) is gated behind human Slack approval; all other outputs are read-only
Error resilience	Two-tier handling: expected failures (malformed LLM JSON) degrade gracefully inline (N8/E); unexpected failures (node crash, API error) are caught globally (EC1/EC2) and alerted to a separate #errors channel with a direct execution link
Consistency of format	Fixed JSON schema per capability (see §PRD Migration: Functional Requirements above), so downstream formatting is deterministic regardless of which LLM run produced the data
Reliability under LLM unpredictability	Observed during testing: the LLM's own sufficient_detail/sufficient_data gatekeeping and status-bucketing were not reliable enough on their own — addressed by adding deterministic overrides (N17d, N26b, N26d) that re-check the model's claims against ground truth before they reach the user, rather than trusting single-pass LLM judgment for consequential branching decisions
Cost/quota awareness	LangSmith free-tier tracing (5,000 traces/month) budgeted against observed usage of ~3–5 traces per agent turn

13. Cross-Cutting Technical Concerns

- **Reliability / error handling:** dedicated error-handling nodes (E, EC1, EC2) catch workflow failures and post to the Slack #errors channel, so a failed run surfaces immediately rather than silently dropping a request.
- **Idempotency:** the Ingestion pipeline is explicitly wipe-and-rebuild (delete the namespace's vectors, then re-insert) rather than incremental upsert — re-running it never leaves duplicate or orphaned vectors, at the cost of a full re-embed each cycle. This can be enhanced in the next iteration to do an incremental upsert — a future enhancement.
- **Grounding integrity over LLM confidence:** the deterministic guardrail layer (N17d, N26d) is the system's main reliability control — it treats every LLM output as a draft to be checked against live data, not a final answer, which is what makes the system safe to run unattended between HITL gates.

Mira — Component Level Design Document

- **Cost / quota:** gpt-4o-mini was chosen as the base model for every agent (and the fine-tuning base) partly for cost — five agents plus a daily Milestone Tracker run on every active session, so per-call cost compounds; embeddings are only recomputed on ingestion runs, not per request, keeping steady-state retrieval cheap.
- **Scalability path to multi-project:** the current build is scoped to a single project's Drive folder, Sheets, and Pinecone namespace. Extending to multiple projects would mean parameterizing the Drive folder ID / Sheet IDs / namespace per project (likely one namespace per project inside the same index, selected at query time) rather than a structural rework — the retrieve-as-tool and deterministic-guardrail patterns generalize as-is.
- **Observability as a first-class concern, not an afterthought:** LangSmith tracing and the DeepEval offline suite were built alongside the agents rather than bolted on after — the Risk Assessor's fine-tuning decision itself came out of evaluation data, not intuition.

14. Open Technical Risks & Future Architecture Considerations

Risk / gap	Why it matters	Possible direction
Webhook entry point is unauthenticated in this build.	Anyone who obtains the URL can invoke Mira and read its responses; the only real trust boundary today is that the URL isn't published	Add an API key or signed-request check on the webhook node before any external-facing use
Single Pinecone namespace / single project scope	The current design assumes one project's data at a time	Namespace-per-project (or metadata-filtered multi-tenant namespace) with the project selected at query time
Wipe-and-rebuild ingestion has a brief inconsistency window	Between the delete and the re-insert completing, a concurrent retrieval could see a partially-populated index	Blue/green namespace swap (build into a new namespace, then repoint retrieval) instead of delete-then-rebuild in place
Fine-tuning is a manual, single-agent process today	Only the Risk Assessor has a fine-tuning pipeline; the other agents rely on prompt iteration alone	If Status Reporter or Planner show similar recurring failure classes in LangSmith evaluation data, extend the same train/evaluate pattern to them
Local n8n instance	Fine for capstone development; not resilient to the host machine going down, and has no built-in horizontal scaling	Migrate to n8n Cloud or a self-hosted, monitored deployment if this moves toward production use
Deterministic guardrails are hand-written per agent	N17–N21, N26, and now N31 (added for the Stakeholder Update agent) are each bespoke Code nodes; every new specialist agent so far has needed its own guardrail written from scratch rather than inheriting a shared one	Factor the "re-derive from live sheet and reconcile" pattern into a shared, parameterized verification node
No load/concurrency testing performed	The T1–T14 baseline dataset (the original 12 plus 2 added for Human-in-the-Loop coverage) and the DeepEval suite test correctness, not behavior under concurrent requests	Add concurrency testing before increasing usage beyond the current single-project, low-volume pattern

15. Langsmith - Evaluation Detail

This section details the dataset names, exact test-case codes, and metric definitions.

Mira — Component Level Design Document

LangSmith (offline evaluation, dataset `mira_Ds_v5` (reference document `mira_Ds_v5.csv`)— 12 test cases: T1–T12 and tracing project chosen as `MiraAgenticSolution`:

- **Hallucination** — scored against the retrieved tool output for that run as ground truth
- **Assertions** — literal Must/Must-Not pass/fail per case
- **Answer Relevance** — wherever answer is populated
- **Bias & Fairness** — scoped to the named-person test cases only

Observations and Fixes

Bug "undefined - undefined" placeholder: T005 is correctly placed under "In progress," not Done which is fine however, the real, visible bug in this row is only the "undefined - undefined" placeholders (appearing in *both* the Done and In Progress lists). This is further presented with Eval Screenshots in the document "Mira-Workflow Architecture Node Reference.docx"

16. Fine Tuning - Detail

DeepEval (separate, targeted suite to fine-tuned Risk Assessor, N16): 9 scenario-labeled test cases (T1–T9) (vague, broad, count, named-new, named-existing) scored with 4 custom GEval metrics — Vague Statement Handling, Risk Groundedness, Requested Count Enforcement, New vs. Existing Risk Routing — plus an optional general Answer Relevancy pass. Run before and after fine-tuning for direct comparison.

Documented findings (from the build's own testing, captured as the rationale for the deterministic guardrail nodes): the LLM's self-reported `sufficient_detail/sufficient_data` gatekeeping and status-category bucketing were not reliable enough to trust on a single pass — concretely, vague statements sometimes still returned `sufficient_detail: true`, and status bucketing occasionally misread a task's own status field. This is the evaluation program's single most actionable finding, and it directly produced the N17d/N26d deterministic-override nodes described above, rather than being fixed by prompt-only iteration alone.

Observations and Fixes

Open, unresolved: bare vague-risk queries (T8). A one-word query like "risks?" still fails the Vague Statement Handling metric (scored 0.11 in the latest DeepEval run) — the agent dumps the full register instead of asking for scope. Confirmed via testing that fine-tuning does not fix this class of failure; it needs a prompt-engineering correction to N16's Step-0 intent classification, not another training pass.

Metric-wording artifact, not a functional defect: the Priya Nair new-risk case (T6/T13) scores low (0.36) on Risk Groundedness because that metric penalizes an empty `risks[]` array / missing `risk_id` on a correctly-routed *new* risk — the routing itself (to `new_risks`, intent `both`) is correct and confirmed working end-to-end through Slack approval.

Mira — Component Level Design Document

17. Baseline Test set

Baseline test dataset (14 test cases to cover Human-in-the-Loop, listed in `Mira-Baseline-Test-Results.docx`): covers a detailed plan (T1), a vague plan (T2), a detailed risk assessment (T3), a vague risk assessment (T4), a data-backed status report (T5), a no-data status report (T6), a top-N risk query (T7), a blocked/at-risk task query (T8), an edge-case 2-week-project plan with no other details (T9), a full status summary (T10), an upcoming-milestones query (T11), and a stakeholder-update-style Sprint 2 summary (T12), a departure-triggered new-risk case requiring Slack approval (T13, Priya Nair), and a second departure-impact case (T14, Arjun Mehta). All 14 were run through the live workflow; T13/T14 both correctly routed through the Slack Approve/Deny gate before the register was written.